1.3.3. Self

The Self programming language was created as a research project by David Ungar and Randall Smith based on a programming concept known as *prototypes*. Being based on prototypes, Self was very different from the popular object-oriented languages of the time (such as C++ and Java) that were based on *classes.* You'll learn more about classes and prototypes in later chapters, but for now, think of classes as being a more rigid and static approach, and prototypes as a more fluid and dynamic approach.

Self also had a different style than the other popular object-oriented languages of the time by preferring a small but powerful set of operations to more numerous and elaborate ones. This style was a direct inspiration in the creation of JavaScript, which took not only the idea of prototypes from Self but also this idea of having a small set of powerful primitive tools as a primary design goal. So, although JavaScript looks more like Java or C++ than it does Self, it has some core ideas taken directly from Self. It looks one way but acts another.

Although JavaScript looks like C (the syntax and to some extent the style and semantics), some of the key ideas (and to some extent the style and semantics) are borrowed from Self and Scheme. What happens when a language has many competing factors?

1.4. Evolving JavaScript

The inherent friction between the competing ideas behind JavaScript's creation is compounded by it now being the most widely used programming language in the world. The social aspect of JavaScript as a programming language has more influencing factors than most other programming languages, and the widespread use of JavaScript serves to amplify all of the influences. This is important in understanding the future of JavaScript because the people who use *any* language (JavaScript or otherwise) are those who shape it over time. To illustrate, consider a brief account of a particular *spoken* language.